CS1026B Assignment 3: Room Booking

Due: **Tuesday, March 17th, 2026 @ 11:59 PM ET**

Change Log

- 2026-03-07 The due date was corrected to March 17th at 11:59PM
- 2026-03-06 Updated example 5 for validate_booking(), previously it showed example 4 again rather than example 5.
- 2026-03-02 Initial post, no changes yet.

Table of Contents

- Late Penalty
- Academic Integrity
- Learning Objectives
- Assignment Overview
  - Part 1: File Formats
  - Part 2: Functional Specification
    - clean_room_number(room_number)
    - load_rooms(file_name)
    - load_bookings(file_name)
    - print_room_info(room_dict, room_number)
    - find_rooms_by_feature(room_dict, feature_tag)
    - find_rooms_by_capacity(room_dict, min_capacity)
    - find_bookings(bookings_list, room_number="", date="")

- remove_booking(bookings_list, room_number, date, hour)
- add_booking(bookings_list, booking)
- validate_booking(room_dict, bookings_list, new_booking)
- under_booked(room_dict, bookings_list, percentage)
- save_bookings(bookings_list, file_name)
- main_menu()
- Part 3: main.py
- Part 4: Starter Templates

- Non-Functional Specification and Other Details
- Assessment
- Submission

⌛ Late Penalty

❗ Late submissions will be given a **zero grade** if you have insufficient late coupons remaining.

⚠️ Any resubmissions past the due date will be considered late and will require late coupons.

Assignments will be accepted up to 3 days late, but **only if you have at least one late coupon remaining for each day late** (see the course syllabus for full details on how late coupons work).

If you submit late and do not have enough late coupons remaining, a **zero grade** will be given for the assignment (there is no partial late mark).

No action is required to submit late. Simply submit late and your late coupons will be applied by the TA during marking.

▲ Back to Top

⚖️ Academic Integrity

By submitting this assignment, you agree that you have read and understood the academic dishonesty policy for this course covered in both the course syllabus and the academic

dishonesty document. This includes:

- Assignments must be completed individually, without any help from others.
- No copying solutions, formulas, files, or written answers from other students.
- No sharing of any part of your work with other students.
- No unauthorized use of generative AI tools (e.g. ChatGPT, Copilot, etc.). You can not use generative AI for this assignment.

> Suspected violations will be reported to the integrity committee and receive a zero grade.

▲ Back to Top

Learning Objectives

1. Read **CSV** and **TSV** files using Python **file I/O**, and apply parsing logic to build **lists** and **dictionaries** from the data.
2. Design and implement **functions** that apply parameters, return values, and decomposition to keep the program modular.
3. Use **lists** to store collections of records and apply iteration patterns to search, filter, and accumulate results.
4. Use **dictionaries** to map keys to structured information and apply dictionary lookup to retrieve room attributes efficiently.
5. Apply **string processing** (splitting, stripping, case normalization, character tests) to clean input data and produce consistent identifiers.
6. Analyze and implement a complex software specification in Python.

▲ Back to Top

Assignment Instructions

> Make sure that you read over the Non-Functional Specification before you start coding. It contains requirements for comments, variable names, and style you need to follow.

For this assignment you will be creating ***one*** Python program comprised of ***two*** Python (`.py`) files:

- `booking.py`: all of your functions should be defined in this file, with no code outside of the functions.
- `main.py`: the code to drive your program and call the functions from `booking.py`

Part 1 will give an overview of the format of the files being used, Part 2 will give a specification for the functions you must implement in `booking.py`, Part 3 will explain the input and output (I/O) required for `main.py`, and Part 4 has starter templates for both `booking.py` and `main.py`. It is recommended that you read each part before you start coding your program.

Introduction

In this assignment you will build a small command-line program in Python that manages room bookings using two data files. One file (`rooms.csv`) lists rooms and their details (capacity and features). The other file (`bookings.tsv`) lists booked events (room, date, hour, number of attendees, and purpose).

Your program will load both files, let a user view room and booking information, search for rooms that meet requirements (like minimum capacity or specific features), and add or remove bookings while enforcing basic rules (for example, you cannot exceed the room's capacity and you cannot double-book the same room for the same date and hour).

You will implement the program using a **functional approach**: the core logic must be broken into a set of standalone functions that take inputs, return outputs, and avoid relying on global variables. Part 2 provides a detailed specification for each required function, and your solution will be assessed against those specifications, so follow them exactly.

▲ Back to Top

Part 1: File Formats

This part outlines the format of the files your program will be reading data from `rooms.csv` and `bookings.tsv`. The format of these files will always remain the same and you can assume they will always have a valid file format, however, the data contained in these files may be different in

each test run by the autograder. You can find an example of each file here: rooms.csv and bookings.tsv.

rooms.csv

This Comma-Separated Values (CSV) formatted file contains information about each room that is available to book. The first row of the file contains a header for each column, the following rows each represent one room and include:

1. The **room number** (a string, for example AHB 1B02).
2. The **building** (a string of text) the room is located in.
3. The room's **maximum capacity** (a positive integer value).
4. A **feature list** describing what the room supports (such as projectors, document cameras, or table layout). The features column stores multiple feature tags in a single cell, separated by the | character.

The following is an example of what the file might contain:

```
1 room_number,building,capacity,features
2 3M 3250,3M Centre,168,dual_projector|document_camera|solstice|hdmi|mic
3 AHB 1B02,Arts & Humanities Building,78,single_projector|document_camer
4 AHB 1B04,Arts & Humanities Building,78,single_projector|document_camer
5 FNB 1200,FIMS & Nursing Building,80,dual_projector|document_camera|pre
6 FNB 1220,FIMS & Nursing Building,80,dual_projector|document_camera|pre
7 HSB 9,Arthur & Sonia Labatt Health Sciences Building,50,dual_projector
8 HSB 11,Arthur & Sonia Labatt Health Sciences Building,50,dual_projecto
```

Note that line 1 contains the header row which does not contain any data. Each column of data is separated by a comma (`,`) and the feature tags are separated by a pipe character (`|`).

Line 2 would contain the data for room `3M 3250`, in the building `3M Centre`, with a max capacity of `168`. The feature tags for this room would be: dual_projector, document_camera, solstice, hdmi, microphone, classroom_computer, presenter_camera, and continuous_tables.

A `rooms.csv` file will always contain at least one room and each room will always have at least two features.

bookings.tsv

This Tab-Separated Values (TSV) formatted file contains the current bookings for each room. Each row represents one booking and includes the **room number** (a string, for example AHB 1B02), the **booking date** (`YYYY-MM-DD`), the **start hour** (an integer hour on a 24-clock between 0 and 23), the **number of attendees** for the event (a positive integer value), and a **short purpose/description** (a string of text). All bookings are for exactly one hour and start on the hour (e.g. 19:00 and not 19:30 or 19:01), multiple consecutive one hour bookings can be made if time slots longer than one hour are needed.

The following is an example of what the file might contain:

```
1  room_number date    hour    num_attendees   purpose
2  3M 3250 2026-03-03  9   100 ENG101 Lecture
3  3M 3250 2026-03-04  9   160 ENG101 Lecture
4  3M 3250 2026-03-05  9   137 ENG101 Lecture
5  3M 3250 2026-03-06  9   140 ENG101 Lecture
6  3M 3250 2026-03-07  9   137 ENG101 Lecture
7  AHB 1B02    2026-03-03  11  78  CS1026 Lab
8  AHB 1B02    2026-03-03  12  70  CS1026 Lab
9  AHB 1B02    2026-03-04  11  70  CS1026 Lab
10 AHB 1B02    2026-03-04  12  78  CS1026 Lab
11 AHB 1B02    2026-03-05  11  68  CS1026 Lab
12 AHB 1B02    2026-03-05  12  78  CS1026 Lab
13 AHB 1B02    2026-03-06  11  66  CS1026 Lab
14 AHB 1B02    2026-03-06  12  66  CS1026 Lab
15 AHB 1B02    2026-03-07  11  67  CS1026 Lab
16 AHB 1B02    2026-03-07  12  75  CS1026 Lab
17 FNB 1200    2026-03-03  9   80  Office Hour Block
18 FNB 1200    2026-03-05  14  80  Midterm Exam
```

Note that line 1 contains the header row which does not contain any data. Each column of data is separated by tab character (`\t`). Not all rooms will have bookings.

Line 2 would contain a booking from room `3M 3250` for March 3rd, 2026 (`2026-03-03`) at 9:00AM (9) for `100` attendees. The description given for the booking is `ENG101 Lecture`.

A `bookings.tsv` file will always contain at least one room booking and will have a value for each column.

▲ Back to Top

Part 2: Functional Specification

Your program must define and implement the functions specified in this part in your `booking.py` file. The function and parameters must be named exactly as given in this part and the parameters must come in the same order. The returned value from each function must be of the correct type and formatted as specified for each function. Your `booking.py` file should only contain function definitions and have no code outside of the functions.

A starter template for the `booking.py` file is given in Part 4.

`clean_room_number(room_number)`

`clean_room_number` takes a single string parameter `room_number` that may contain spaces, punctuation, or mixed case (for example `"AHB 1B02"`, `"ahb-1b02"`, or `" n s c 1 "`). The function returns a new string where the input has been **converted to uppercase** and **all non-alphanumeric characters have been removed**.

Example:

```
from booking import *

x = clean_room_number("AHB 1B02")
y = clean_room_number("ahb-1b02")
z = clean_room_number(" nsc1 ")
print(x)
print(y)
print(z)
```

Output:

```
AHB1B02
AHB1B02
NSC1
```

`load_rooms(file_name)`

The `load_rooms` function reads the rooms CSV file and builds a dictionary of room information keyed by a cleaned room number (cleaned using the clean_room_number function).

Parameter

- `file_name`: A string containing the filename of the CSV file to load. You can assume this file is in the same directory as your program and is formatted the same as the rooms CSV file but may have any filename.

Return

This function returns a **nested dictionary**. In the outer dictionary keys are room numbers cleaned with the clean_room_number function (such as `"NSC1"` or `"AHB1B02"`) and values are a dictionary that describes the room with the following key/value pairs *(with each key being a string)*:

- **building:** A *string value* containing the full name of the building the room is located in such as `"3M Centre"` or `"Arts & Humanities Building"`.
- **capacity:** A *positive integer value* containing the maximum capacity for the room (how many people can fit in the room).
- **features:** A *list of strings* that has an element for each feature tag that describes the features of the room such as `"document_camera"` or `"dual_projector"`.

Assumptions

You can assume that the file exists and is formatted correctly. You can also assume that the file will always contain at least one room and each room will have at least two feature tags. No error checking or validation on the file is needed but room numbers must be cleaned with the clean_room_number function.

Example

This example shows the result of calling the `load_rooms` function on the two_rooms.csv file which contains only two rooms:

```
from booking import *

rooms = load_rooms("two_rooms.csv")
print(rooms)
```

The returned dictionary would be:

```
{'3M3250': {'building': '3M Centre',
            'capacity': 168,
            'features': ['dual_projector',
                         'document_camera',
                         'solstice',
                         'hdmi',
                         'microphone',
                         'classroom_computer',
```

```
                          'presenter_camera',
                          'continuous_tables']},
 'AHB1B02': {'building': 'Arts & Humanities Building',
             'capacity': 78,
             'features': ['single_projector',
                          'document_camera',
                          'classroom_computer',
                          'solstice',
                          'hdmi',
                          'vga',
                          'microphone',
                          'continuous_tables']}}
```

Note that this output would be on one line in the console, the above is formatted to make it easier to read.

`load_bookings(file_name)`

The `load_bookings` function reads the bookings TSV file and builds a **list** of dictionaries which each describe a booking of a room (one dictionary per booking).

Parameter

- `file_name`: A string containing the filename of the TSV file to load. You can assume this file is in the same directory as your program and is formatted the same as the bookings TSV file but may have any filename.

Return

This function returns a **list** of **dictionaries**. Each dictionary in the list describes an individual 1 hour room booking with the following key/value pairs *(with each key being a string)*:

- **room_number**: A *string value* containing the room number after being cleaned with the clean_room_number function.
- **date**: A *string value* containing the date the room was booked for in the `YYYY-MM-DD` format such as `"2026-04-12"`.
- **hour**: A *non-negative integer value* between 0 and 23 (inclusive) that represent the hour of the day the room is booked for using a 24 hour clock. For example `13` would be `1PM`.
- **num_attendees**: A *positive integer value* representing the number of persons attending the event the room is booked for.
- **purpose**: A *string value* containing a short description of the purpose of the room booking such as `"Faculty Teaching Demo"`.

Assumptions

You can assume that the file exists and is formatted correctly. You can also assume that the file will always contain at least one room booking with a valid date and hour value. No error checking or validation on the file is needed but room numbers must be cleaned with the clean_room_number function.

Example

This example shows the result of calling the `load_bookings` function on the two_bookings.tsv file which contains only two room bookings:

```
from booking import *

bookings = load_bookings("two_bookings.tsv")
print(bookings)
```

The returned list would be:

```
[{'room_number': '3M3250',
  'date': '2026-03-03',
  'hour': 9,
  'num_attendees': 100,
  'purpose': 'ENG101 Lecture'},
 {'room_number': 'AHB1B02',
  'date': '2026-03-03',
  'hour': 11,
  'num_attendees': 78,
  'purpose': 'CS1026 Lab'}]
```

Note that this output would be on one line in the console, the above is formatted to make it easier to read.

`print_room_info(room_dict, room_number)`

This function takes a room dictionary (as created by the load_rooms function) and a room number and prints the information for that room in an easy to read format as shown in the given example. The function has no returned value.

Parameters

- `room_dict`: A nested dictionary of rooms as created by the load_rooms function. You can assume it is correctly formatted.
- `room_number`: A clean room number with no spaces of special characters such as `"3M3250"` or `"AHB1B02"`. You can assume that the room number is contained in the given dictionary.

Example

This example prints a hard coded room dictionary, typically room dictionaries would be created by the load_rooms function.

```
from booking import *

rooms = {'3M3250': {'building': '3M Centre',
                    'capacity': 168,
                    'features': ['dual_projector',
                                 'document_camera',
                                 'solstice',
                                 'hdmi',
                                 'microphone',
                                 'classroom_computer',
                                 'presenter_camera',
                                 'continuous_tables']},
         'AHB1B02': {'building': 'Arts & Humanities Building',
                     'capacity': 78,
                     'features': ['single_projector',
                                  'document_camera',
                                  'classroom_computer',
                                  'solstice',
                                  'hdmi',
                                  'vga',
                                  'microphone',
                                  'continuous_tables']}}

print_room_info(rooms, "AHB1B02")
```

Output

```
Room: AHB1B02
Building: Arts & Humanities Building
Capacity: 78
Features:
        single_projector
```

```
        document_camera
        classroom_computer
        solstice
        hdmi
        vga
        microphone
        continuous_tables
```

`find_rooms_by_feature(room_dict, feature_tag)`

The `find_rooms_by_feature` function takes a room dictionary (as created by the load_rooms function) and single feature tag and returns a **list** of room numbers that have this feature.

Parameters

- `room_dict`: A nested dictionary of rooms as created by the load_rooms function. You can assume it is correctly formatted.
- `feature_tag`: A *string value* that contains a single feature tag such as `"single_projector"` or `"hdmi"`. The feature tag may or may not exist in the room dictionary but will always be lower case.

Return

This function returns a **list of string values**, each containing a clean room number from the `room_dict` that has the feature given by `feature_tag`. If no rooms have the given feature, this function should return an empty list `[]`. The list can be in any order.

Example

This example uses the simple_rooms.csv file.

```python
from booking import *
rooms = load_rooms("simple_rooms.csv")

has_dual_projector = find_rooms_by_feature(rooms, "dual_projector")
has_vga = find_rooms_by_feature(rooms, "vga")

print("Dual Projectors: ", has_dual_projector)
print("VGA: ", has_vga)
```

Output

```
Dual Projectors:  ['3M3250', 'FNB1200', 'FNB1220', 'HSB9', 'HSB11']
VGA:  ['AHB1B02', 'AHB1B04']
```

`find_rooms_by_capacity(room_dict, min_capacity)`

The `find_rooms_by_capacity` function takes a room dictionary (as created by the load_rooms function) and a minimum capacity and returns a list of room numbers that have this capacity or greater. The idea is that this function can help answer the question *"Can this room fit X number of people?"* where `min_capacity` is X.

Parameters

- `room_dict`: A nested dictionary of rooms as created by the load_rooms function. You can assume it is correctly formatted.
- `min_capacity`: A *positive integer value* that contains the minimum capacity a room must have to be included in the returned list.

Return

This function returns a **list of string values**, each containing a clean room number from the `room_dict` that has a capacity value equal to or larger than `min_capacity`. If no rooms have the required capacity, this function should return an empty list `[]`. The list can be in any order.

Example

This example uses the rooms.csv file.

```python
from booking import *
rooms = load_rooms("rooms.csv")

has_241 = find_rooms_by_capacity(rooms, 241)
has_600 = find_rooms_by_capacity(rooms, 600)
has_790 = find_rooms_by_capacity(rooms, 790)

print("Min 241: ", has_241)
print("Min 600: ", has_600)
print("Min 790: ", has_790)
```

Output

```
Min 241:  ['AHB1R40', 'HSB40', 'MC110', 'NSC1', 'NSC145', 'NCB101',
Min 600:  ['NSC145', 'NCB101']
Min 790:  []
```

`find_bookings(bookings_list, room_number="", date="")`

The `find_bookings` function returns a list of bookings for a given room number and/or date contained in the given bookings_list (as created by the load_bookings function). In simple terms, this function allows us to filter a booking list by a specified room number and/or date.

Parameters

- `bookings_list`: A list of booking dictionaries as created by the load_bookings function. You can assume it is correctly formatted.
- `room_number`: Either a *string value* that contains a single clean room number (such as `"3M3250"` or `"AHB1B02"`) or an empty string (`""`). If a room number is given, the list is filtered such that it only includes bookings for that room number. If an empty string is given, this parameter is ignored and the list is not filtered by room number. If a room number is given but does not occur in any bookings `room_number`, an empty list (`[]`) should be returned.
- `date`: Either a *string value* containing a date in the `YYYY-MM-DD` format (such as `2026-04-12`) or an empty string (`""`). If a date is given, the list is filtered such that it only includes bookings for the given date. If an empty string is given, this parameter is ignored and the list is not filtered by date. If a date is given, you can assume the given date is valid and in the correct format. If the date is given but there are no bookings for this date, an empty list (`[]`) should be returned.

Return

This function returns a **list of room bookings** from `bookings_list` filtered by the given `room_number` and `date`. If both `room_number` and `date` are empty strings, a shallow copy of the `bookings_list` should be returned with all bookings. If both a `room_number` and `date` are given, the list should be filtered by both (i.e. only bookings for this room and on this given date). If no bookings match the given parameters, an empty list (`[]`) should be returned.

Example

This example uses the simple_bookings.tsv file.

```python
from booking import *
bookings = load_bookings("simple_bookings.tsv")

for_fnb1200 = find_bookings(bookings, "FNB1200")
for_the_7th = find_bookings(bookings, date="2026-03-07")
both_ahb1B02_and_3rd = find_bookings(bookings, "AHB1B02", "2026-03-03"

print("for_fnb1200:")
print(for_fnb1200)
print("\nfor_the_7th:")
print(for_the_7th)
print("\nboth_ahb1B02_and_3rd:")
print(both_ahb1B02_and_3rd)
```

Output

```
for_fnb1200:
[{'room_number': 'FNB1200',
  'date': '2026-03-03',
  'hour': 9,
  'num_attendees': 80,
  'purpose': 'Office Hour Block'},
 {'room_number': 'FNB1200',
  'date': '2026-03-05',
  'hour': 14,
  'num_attendees': 80,
  'purpose': 'Midterm Exam'}]

for_the_7th:
[{'room_number': '3M3250',
  'date': '2026-03-07',
  'hour': 9,
  'num_attendees': 137,
  'purpose': 'ENG101 Lecture'},
 {'room_number': 'AHB1B02',
  'date': '2026-03-07',
  'hour': 11,
  'num_attendees': 67,
  'purpose': 'CS1026 Lab'},
 {'room_number': 'AHB1B02',
```

```
   'date': '2026-03-07',
   'hour': 12,
   'num_attendees': 75,
   'purpose': 'CS1026 Lab'}]

 both_ahb1B02_and_3rd:
 [{'room_number': 'AHB1B02',
   'date': '2026-03-03',
   'hour': 11,
   'num_attendees': 78,
   'purpose': 'CS1026 Lab'},
  {'room_number': 'AHB1B02',
   'date': '2026-03-03',
   'hour': 12,
   'num_attendees': 70,
   'purpose': 'CS1026 Lab'}]
```

Note that this output for each list would be on one line in the console, the above is formatted to make it easier to read.

`remove_booking(bookings_list, room_number, date, hour)`

The `remove_booking` removes a booking from the given `bookings_list` that has a matching `room_number`, `date`, and `hour` (all must match exactly to be removed). The booking list should be modified in-place and not returned from the function. The function should return `True` if a booking was removed and `False` if the booking was not found in `bookings_list`.

Parameters

- `bookings_list`: A list of booking dictionaries as created by the load_bookings function. You can assume it is correctly formatted.
- `room_number`: A *string value* that contains a single clean room number (such as `"3M3250"` or `"AHB1B02"`). You can assume the room number is valid, but may not be in the `bookings_list`.
- `date`: A *string value* containing a date in the `YYYY-MM-DD` format (such as `2026-04-12`). You can assume the given date is valid and in the correct format, but may not be in the `bookings_list`.
- `hour`: A *non-negative integer value* between 0 and 23 representing the hour on a 24-hour clock the room was booked for (e.g. 13 would be for 1PM). You can assume the given hour is valid, but may not be in the `bookings_list`.

Return

This function should return `True` if a booking was found and removed with a matching `room_number`, `date`, and `hour` (all must match exactly). Otherwise, if no matching booking was found in `bookings_list` nothing should be removed and a `False` value should be returned.

Example 1

This example uses the four_bookings.tsv file.

```
from booking import *
booking = load_bookings("four_bookings.tsv")

output = remove_booking(booking, "AHB1B02", "2026-03-03", 12)

print("Output:", output)
print("\nBookings Left:")
print(booking)
```

Output 1

```
Output: True

Bookings Left:
[{'room_number': 'AHB1B02',
  'date': '2026-03-03',
  'hour': 11,
  'num_attendees': 78,
  'purpose': 'CS1026 Lab'},
 {'room_number': 'AHB1B02',
  'date': '2026-03-04',
  'hour': 11,
  'num_attendees': 78,
  'purpose': 'Midterm Exam'},
 {'room_number': 'FNB1200',
  'date': '2026-03-03',
  'hour': 11,
  'num_attendees': 78,
  'purpose': 'CS1032 Lab'}]
```

Note that this output for the resulting booking list would be on one line in the console, the above is formatted to make it easier to read.

Example 2

This example uses the four_bookings.tsv file and attempts to remove a booking that does not exist.

```
from booking import *
booking = load_bookings("four_bookings.tsv")

# Should be no bookings on 2026-03-07
output = remove_booking(booking, "AHB1B02", "2026-03-07", 12)

print("Output:", output)
print("\nBookings Left:")
print(booking)
```

Output 2

```
Output: False

Bookings Left:
[{'room_number': 'AHB1B02',
  'date': '2026-03-03',
  'hour': 11,
  'num_attendees': 78,
  'purpose': 'CS1026 Lab'},
 {'room_number': 'AHB1B02',
  'date': '2026-03-03',
  'hour': 12,
  'num_attendees': 70,
  'purpose': 'CS2212 Lecture'},
 {'room_number': 'AHB1B02',
  'date': '2026-03-04',
  'hour': 11,
  'num_attendees': 78,
  'purpose': 'Midterm Exam'},
 {'room_number': 'FNB1200',
  'date': '2026-03-03',
  'hour': 11,
  'num_attendees': 78,
  'purpose': 'CS1032 Lab'}]
```

Note that this output for the resulting booking list would be on one line in the console, the above is formatted to make it easier to read.

`add_booking(bookings_list, booking)`

This function simply adds the given `booking` dictionary to the `bookings_list`. You can assume that `booking` is a properly formatted booking dictionary and can be added directly to the `bookings_list` without validation. `bookings_list` should be modified in-place and this function should have no return.

Example

This example uses the two_bookings.tsv file.

```
from booking import *
bookings = load_bookings("two_bookings.tsv")

new_booking = {'room_number': 'FNB1200',
               'date': '2026-03-03',
               'hour': 11,
               'num_attendees': 78,
               'purpose': 'CS1032 Lab'}

add_booking(bookings, new_booking)
print("Bookings after addition:")
print(bookings)
```

Output

```
Bookings after addition:
[{'room_number': '3M3250',
  'date': '2026-03-03',
  'hour': 9,
  'num_attendees': 100,
  'purpose': 'ENG101 Lecture'},
 {'room_number': 'AHB1B02',
  'date': '2026-03-03',
  'hour': 11,
  'num_attendees': 78,
  'purpose': 'CS1026 Lab'},
```

```
 {'room_number': 'FNB1200',
  'date': '2026-03-03',
  'hour': 11,
  'num_attendees': 78,
  'purpose': 'CS1032 Lab'}]
```

Note that this output for the dictionary would be on one line in the console, the above is formatted to make it easier to read.

`validate_booking(room_dict, bookings_list, new_booking)`

The `validate_booking` function checks if a new booking dictionary (`new_booking`) is valid. If the booking is valid, your function should return a `True` value. If the booking is invalid your function should both return a `False` value and print an error message.

Your function must check the following in this order:

1. The new booking contains a room number that exists (i.e. it is in the `room_dict`). Output `"Invalid room!"` if it does not.
2. The new booking contains a date that is in the `YYYY-MM-DD` format, that is four digits, a dash, two digits, a dash, and then two digits. You do not have to check that the date makes sense otherwise (e.g. accept `2026-32-99` as a valid date). If the date format is invalid your function should output `"Invalid date!"`.
3. The room is not already booked at the given date and hour. If it is, your function should output `"Room already booked at this date and time!"`.
4. The capacity of the room being booked is large enough to fit the number of attendees listed in the new booking. If the capacity is smaller than the number of attendees, your function should output `"Room does not have required capacity!"`.

If the new booking is invalid in more than one way, it should only output the first error message that matches in the above list. You only need to check for the above issues.

Parameters

- `room_dict`: A nested dictionary of rooms as created by the load_rooms function. You can assume it is correctly formatted.
- `bookings_list`: A list of booking dictionaries as created by the load_bookings function. You can assume it is correctly formatted.
- `new_booking`: A new booking dictionary to be validated. You can assume that it has all of the required keys but the values may be invalid.

Example 1

This example uses the rooms.csv and bookings.tsv files.

```
from booking import *

rooms = load_rooms("rooms.csv")
bookings = load_bookings("bookings.tsv")

# MC387 is not in the rooms dict.
my_new_booking = {'room_number': 'MC387',
                  'date': '2026-03-03',
                  'hour': 9,
                  'num_attendees': 100,
                  'purpose': 'CS1026 Lecture'}

output = validate_booking(rooms, bookings, my_new_booking)
print("Output (return from function):", output)
```

Output 1

```
Invalid room!
Output (return from function): False
```

Example 2

This example uses the rooms.csv and bookings.tsv files.

```
from booking import *

rooms = load_rooms("rooms.csv")
bookings = load_bookings("bookings.tsv")

# The date is not in the YYYY-MM-DD format.
my_new_booking = {'room_number': '3M3250',
                  'date': '03/03/2026',
                  'hour': 9,
                  'num_attendees': 100,
                  'purpose': 'CS1026 Lecture'}

output = validate_booking(rooms, bookings, my_new_booking)
print("Output (return from function):", output)
```

Output 2

```
Invalid date!
Output (return from function): False
```

Example 3

This example uses the rooms.csv and bookings.tsv files.

```
from booking import *

rooms = load_rooms("rooms.csv")
bookings = load_bookings("bookings.tsv")

# The room is already booked at this date and hour.
my_new_booking = {'room_number': '3M3250',
                  'date': '2026-03-03',
                  'hour': 9,
                  'num_attendees': 100,
                  'purpose': 'CS1026 Lecture'}

output = validate_booking(rooms, bookings, my_new_booking)
print("Output (return from function):", output)
```

Output 3

```
Room already booked at this date and time!
Output (return from function): False
```

Example 4

This example uses the rooms.csv and bookings.tsv files.

```
from booking import *

rooms = load_rooms("rooms.csv")
bookings = load_bookings("bookings.tsv")

# The number of attendees is too large to fit in this room.
my_new_booking = {'room_number': '3M3250',
                  'date': '2026-03-10',
                  'hour': 9,
```

```
                    'num_attendees': 170,
                    'purpose': 'CS1026 Lecture'}

output = validate_booking(rooms, bookings, my_new_booking)
print("Output (return from function):", output)
```

Output 4

```
Room does not have required capacity!
Output (return from function): False
```

Example 5

This example uses the rooms.csv and bookings.tsv files.

```
from booking import *

rooms = load_rooms("rooms.csv")
bookings = load_bookings("bookings.tsv")

# This should be a valid booking.
my_new_booking = {'room_number': '3M3250',
                  'date': '2026-03-10',
                  'hour': 9,
                  'num_attendees': 100,
                  'purpose': 'CS1026 Lecture'}

output = validate_booking(rooms, bookings, my_new_booking)
print("Output (return from function):", output)
```

Output 5

```
Output (return from function): True
```

`under_booked(room_dict, bookings_list, percentage)`

The `under_booked` function should return a list of tuples (see the Return section below) that represent bookings for rooms that are under booked. Under booked is defined as being below the given `percentage` in terms of capacity. For example, if a room had a maximum capacity of

100 and the booking had 25 attendees, this room would be at 25% capacity. If `percentage` was `0.3` (for 30%) this room would be considered to be under booked (as 25% < 30%).

Parameters

- `room_dict`: A nested dictionary of rooms as created by the load_rooms function. You can assume it is correctly formatted.
- `bookings_list`: A list of booking dictionaries as created by the load_bookings function. You can assume it is correctly formatted.
- `percentage`: A *floating point value* between 0 and 1 (inclusive) that represents the percentage threshold for a room to be considered under booked (i.e. a booking with a capacity percentage less than this is under booked).

Return

This function returns a **list of tuples**, where each tuple represents a booking from the `bookings_list` that is considered to be under booked. If no bookings are under booked, this function will return an empty list (`[]`). Each tuple in the list should contain the following values in this order:

1. The clean room number as a string value (such as `"3M3250"` or `"AHB1B02"`).
2. The date of the booking (a string with the format `YYYY-MM-DD`).
3. The hour of the booking, a non-negative integer value, between 0 and 23 (inclusive).
4. The current capacity percentage of the booking as a floating point value between 0 and 1 (inclusive). For example if a booking had 75 attendees and the maximum capacity for the room was 100, this value would be 0.75. Do not round the value.

Example

This example uses the four_rooms.csv and four_bookings.tsv files.

```
from booking import *
rooms = load_rooms("four_rooms.csv")
bookings = load_bookings("four_bookings.tsv")

# Anything under 98% is under booked in this case.
under = under_booked(rooms, bookings, 0.98)

print(under)
```

Output

```
[('AHB1B02', '2026-03-03', 12, 0.8974358974358975),
```

```
('FNB1200', '2026-03-03', 11, 0.975)]
```

Note that this output would be on one line in the console, the above is formatted to make it easier to read.

`save_bookings(bookings_list, file_name)`

The `save_bookings` function saves the given bookings list (as created by the load_bookings function) to the given file name. The format used in this file should be the same as the bookings TSV file described in Part 1. You can assume the `bookings_list` and `file_name` are valid and that `bookings_list` contains at least one booking. The function does not return any value.

Parameters

- `bookings_list`: A list of booking dictionaries as created by the load_bookings function. You can assume it is correctly formatted.
- `file_name`: A *string value* containing the file name to save the bookings into such as `"new_bookings.tsv"`. You can assume it is a valid file name that does not yet exist, in the same directory as your program.

Example

This example creates a new file named `new_bookings.tsv`.

```
from booking import *

my_bookings = [{'room_number': 'FNB1200',
                'date': '2026-03-03',
                'hour': 11,
                'num_attendees': 78,
                'purpose': 'CS1032 Lab'},
               {'room_number': 'MC17',
                'date': '2026-03-10',
                'hour': 17,
                'num_attendees': 15,
                'purpose': 'Makeup Exam'},
               {'room_number': 'NSC1',
                'date': '2026-05-20',
                'hour': 19,
                'num_attendees': 300,
                'purpose': 'Final Exam'}]

```

```
save_bookings(my_bookings, "new_bookings.tsv")
```

Output

This example would not output anything to the console but the following would be the contents of the newly created `new_bookings.tsv` file:

```
room_number date    hour    num_attendees   purpose
FNB1200 2026-03-03  11  78  CS1032 Lab
MC17    2026-03-10  17  15  Makeup Exam
NSC1    2026-05-20  19  300 Final Exam
```

`main_menu()`

The code for this function is given to you in [the starter template found in Part 4](). This function displays a text-based menu (see the example below) to the user and accepts input (a number between 1 and 9), validates the input, and returns it if valid. If invalid, the function continues to ask the user for input until valid input is given.

Return

An integer value between 1 and 9, representing the user's menu selection.

Example Output

The following is an example of the output given from calling the `main_menu` function.

```
Select an option:
1) Display room information.
2) Search for room by feature.
3) Search for room by capacity.
4) Search for booking.
5) Make booking.
6) Remove booking.
7) Find under booked rooms.
8) Save bookings.
9) Exit.

Input selection (1-9): 0
Invalid input! Try again.
```

```
Input selection (1-9): 10
Invalid input! Try again.

Input selection (1-9): 7
```

The green text shown above is the user's input. The value returned form `main_menu` would be **7**.

▲ Back to Top

Part 3: main.py

For this last part of the assignment, **you must complete the `main.py` starter template found in Part 4**. This program should implement each option given in the main menu (as output by the main_menu function) by taking advantage of the functions you created in `booking.py` in Part 2. You may also add any additional helper functions you feel are necessary.

The files used in the examples in this part are rooms.csv and bookings.tsv.

Menu Options

The following subsections describe how your program should handle each menu option.

Option 1: Display room information
The program should ask the user for a room number, clean that input using the clean_room_number function, and then use the print_room_info function to display the room information if the room exists. Otherwise if the room does not exist, it should output `"Invalid room number!"`.

Example *(invalid room number, not in the room dictionary)*:

```
Input selection (1-9): 1

Input room number: MC387
Invalid room number!
```

Example *(valid room number, contained in the room dictionary)*:

```
Input selection (1-9): 1

Input room number: mc 17
Room: MC17
Building: Middlesex College
Capacity: 48
Features:
        single_projector
        presenter_camera
        classroom_computer
        solstice
        hdmi
        microphone
        theatre_tab_arm
```

Option 2: Search for room by feature

The program should ask the user for a feature tag to search for and output the list of rooms that have that feature (taking advantage of the find_rooms_by_feature function) or the text `"No rooms found!"` if no rooms have that feature. You do not have to validate or clean the text input by the user for the feature tag.

Example *(invalid feature tag, no rooms have that feature)*:

```
Input selection (1-9): 2

Input feature to search for: banana
No rooms found!
```

Example *(valid feature tag)*:

```
Input selection (1-9): 2

Input feature to search for: vga
Matching rooms: AHB1B02, AHB1B04, AHB1B06, AHB1B08, AHB1R40, AHB2B02
```

Option 3: Search for room by capacity

The program should ask the user for a minimum capacity to search for and output the list of rooms that have that capacity or greater (taking advantage of the find_rooms_by_capacity function) or the text `"No rooms found!"` if no rooms have at least that capacity. You do not have to validate the input and can assume the user input an integer value.

Example *(no rooms have at least that capacity)*:

```
Input selection (1-9): 3

Input minimum capacity to search for: 1000
No rooms found!
```

Example *(rooms have that capacity or more)*:

```
Input selection (1-9): 3

Input minimum capacity to search for: 300
Matching rooms: AHB1R40, HSB40, MC110, NSC1, NSC145, NCB101
```

Option 4: Search for booking

The program should ask the user for a room number and a date and then find all current bookings that match that room and date (taking advantage of the find_bookings function). The room number input by the user should be cleaned using the clean_room_number function. If no bookings are found, the text `"No bookings found!"` should be output, otherwise the details of each booking should be output as shown in the examples. You can assume the date input by the user is valid or an empty string.

Example *(only room number given)*:

```
Input selection (1-9): 4

Input room number (or leave empty for all rooms): n S c - 1
Input date (or leave empty for all dates):
Room: NSC1
Date: 2026-03-03
Hour: 10
```

```
Attendees: 356
Purpose: CS1026 Lecture
------------------------------
Room: NSC1
Date: 2026-03-04
Hour: 10
Attendees: 189
Purpose: Guest Science Talk
------------------------------
Room: NSC1
Date: 2026-03-05
Hour: 10
Attendees: 353
Purpose: CS1026 Lecture
------------------------------
Room: NSC1
Date: 2026-03-06
Hour: 10
Attendees: 239
Purpose: Guest Science Talk
------------------------------
Room: NSC1
Date: 2026-03-07
Hour: 10
Attendees: 242
Purpose: Guest Science Talk
------------------------------
```

Example *(both room number and date given)*:

```
Input selection (1-9): 4

Input room number (or leave empty for all rooms): nsc 1
Input date (or leave empty for all dates): 2026-03-03
Room: NSC1
Date: 2026-03-03
Hour: 10
Attendees: 356
```

```
Purpose: CS1026 Lecture
-------------------------------
```

Example *(no matching bookings)*:

```
Input selection (1-9): 4

Input room number (or leave empty for all rooms): NSC1
Input date (or leave empty for all dates): 2027-06-04
No bookings found!
```

Option 5: Make booking

The program should ask the user for a room number, date, hour, number of attendees, and purpose for a new room booking. The room number should be cleaned using the clean_room_number function and the new booking should be validated with the validate_booking function. If the booking is valid, it should be added to the booking list using the add_booking function and the text `"Booked."` should be output. If the booking is not valid, the text `"Booking not made!"` should be output and the new booking should not be added to the booking list.

Example *(invalid booking, room does not have required capacity)*:

```
Input selection (1-9): 5

Input room number to book: TC 141
Input date to book: 2026-05-20
Input hour to book: 17
Input number of attendees: 400
Input purpose of booking: Final Exam
Room does not have required capacity!
Booking not made!
```

Example *(valid room booking)*:

```
Input selection (1-9): 5

Input room number to book: TC 141
Input date to book: 2026-05-20
Input hour to book: 17
```

```
Input number of attendees: 200
Input purpose of booking: Final
Booked.
```

The booking should also be added to the booking list, `bookings`.

Option 6: Remove booking

The program should ask the user for a room number, date, and hour for an existing booking to be removed. The room number should be cleaned using the clean_room_number function and any matching bookings in the booking list should be removed using the remove_booking function. If the booking was removed successfully the text `"Booking removed."` should be output, otherwise the text `"Booking not found!"` should be output. You can assume the given date is in the correct format and the hour is a valid integer (no need to validate these).

Example *(a matching booking was removed)*:

```
Input selection (1-9): 6

Input room number: NSC 1
Input date: 2026-03-07
Input hour: 10
Booking removed.
```

The matching booking would also be removed from the bookings list, `bookings`.

Example *(no bookings match the input values)*:

```
Input selection (1-9): 6

Input room number: MC387
Input date: 2026-03-07
Input hour: 10
Booking not found!
```

Option 7: Find under booked rooms

The program should use the under_booked function to print a report of all current bookings in the `bookings` list that are under **20%** capacity. The report should include the room number, date, hour, and percent capacity for each matching booking. The percent capacity should be displayed

with one decimal point.

Example:

```
Input selection (1-9): 7

Under booked rooms:
Room       Date          Hour   Percent Capacity
FNB2210    2026-03-03    15     11.9%
FNB2220    2026-03-03    17     17.2%
HSB240     2026-03-03    15     16.7%
NCB117     2026-03-03    12     16.4%
FNB1200    2026-03-04    9      6.2%
MC17       2026-03-04    13     18.8%
FNB1200    2026-03-05    9      11.2%
HSB236     2026-03-06    14     18.7%
MC17       2026-03-06    13     14.6%
FNB2210    2026-03-07    15     19.0%
```

Option 8: Save bookings

The program should ask the user for a file name and use the save_bookings function to save the current bookings list, `bookings`, to a file with the given file name. No validation of the file name is required.

Example:

```
Input selection (1-9): 8

Input file name to save bookings in: my_new_bookings.tsv
Saved!
```

A new file named `my_new_bookings.tsv` would also be created in the current directory with a TSV formatted copy of the `bookings` list.

Option 9: Exit

If this option is selected, the program should simply terminate and no more output given.

▲ Back to Top

Part 4: Starter Templates

> ⚠ Keep `main.py`, `booking.py`, and your CSV and TSV files (e.g. `rooms.csv` and `bookings.tsv`) in the same folder/directory. If they are in a different folder/directory, the data files may not be found by your program.

You should make use of the starter templates given in this section for your `booking.py` and `main.py` Python files. It is important to not include any code outside of the function definitions given in `booking.py` or the autograder may not grade your program correctly. Include any code testing your functions in `main.py` or another file that imports `booking.py` and not in `booking.py` directly.

booking.py

```python
"""
Starter template for booking.py.

You must replace this comment and fill in the details of each function
except for main_menu() which is provided for you. Make sure you remove
the pass keyword after you fill in the code for that function.

See Part 2 of the assignment document for details on the parameters,
return, and specification for each function.

There should be no code in this file outside of any of these function
definitions.
"""


def clean_room_number(room_number):
    pass


def load_rooms(file_name):
    pass


def load_bookings(file_name):
```

```
    pass


def print_room_info(room_dict, room_number):
    pass


def find_rooms_by_feature(room_dict, feature_tag):
    pass


def find_rooms_by_capacity(room_dict, min_capacity):
    pass


def find_bookings(bookings_list, room_number="", date=""):
    pass


def remove_booking(bookings_list, room_number, date, hour):
    pass


def add_booking(bookings_list, booking):
    pass


def validate_booking(room_dict, bookings_list, new_booking):
    pass


def under_booked(room_dict, bookings_list, percentage):
    pass


def save_bookings(bookings_list, file_name):
    pass


def main_menu():
    print("\nSelect an option:")
    print("1) Display room information.")
    print("2) Search for room by feature.")

```

```
    print("3) Search for room by capacity.")
    print("4) Search for booking.")
    print("5) Make booking.")
    print("6) Remove booking.")
    print("7) Find under booked rooms.")
    print("8) Save bookings.")
    print("9) Exit.")

    while True:
        selection = input("\nInput selection (1-9): ")
        if selection.isdigit() and 1 <= int(selection) <= 9:
            break
        else:
            print("Invalid input! Try again.")

    return int(selection)
```

main.py

This program loads `rooms.csv` and `bookings.tsv` from the current directory/folder. You can find an example of these files here: rooms.csv and bookings.tsv.

```
"""
Starter template for main.py.

You must replace this comment and fill in the missing details in this
file to make the program work as described in Part 3 of the assignment
document.

Replace comments such as "# Code for selection 1 goes here." with code
to implement that feature and remove the pass keyword after you have
added your code to that if/elif statement block.

Also add inline comments as needed to this file.
"""

from booking import *

rooms = load_rooms("rooms.csv")
bookings = load_bookings("bookings.tsv")

while True:
    selection = main_menu()
```

```
    if selection == 1:
        # Code for selection 1 goes here.
        pass
    elif selection == 2:
        # Code for selection 2 goes here.
        pass
    elif selection == 3:
        # Code for selection 3 goes here.
        pass
    elif selection == 4:
        # Code for selection 4 goes here.
        pass
    elif selection == 5:
        # Code for selection 5 goes here.
        pass
    elif selection == 6:
        # Code for selection 6 goes here.
        pass
    elif selection == 7:
        # Code for selection 7 goes here.
        pass
    elif selection == 8:
        # Code for selection 8 goes here.
        pass
    elif selection == 9:
        break
```

▲ Back to Top

Non-Functional Specification and Other Details

In addition to the other requirements given in the previous sections, your program must also fulfill the following requirements:

- **Document each file:** You MUST provide a comment at the top of **all** Python files (files ending in .py) that contains 1) your full name, 2) your student number, 3) your uwo username (the same one you use to log in to OWL), 4) the date the file was created, and 5) a description of what this specific file does. This must be provided at the top of all files,

and the description should be updated to describe the specific file it is in (it can not be the same in all files).

- **Other comments throughout your code:** You must also include comments in your code (as well as the given code in `main.py`) for any lines that are not easily understood. You must use good judgment in what lines require comments, the marker may remove marks for over or under commenting.
- **Comment each function:** Before each function in `booking.py` add a comment explaining what this function does, its parameters, and what value it returns (if any).
- **Do not import any modules (other than your own booking module):** for this assignment there should be no need to import any modules.
- **Match the example text:** The text contained in your messages and prompts to the user should match the examples given and all input must be taken in the same order.
- **Use the starter templates:** You must use the starter templates given in Part 4 and follow the specification given in Part 2. You may not change the names of functions, their parameters, or what values they return.
- **Use meaningful variable names:** Your variable names should be meaningful, clearly state the values they store, and use the `snake_case` name format. Don't use variable names that are simply letters such as "t" or "W".
- **Do not use global variables:** All communication with the functions in `booking.py` must occur through the function's parameters and return. You may not use global variables to send values between functions or to `main.py`. You may not use the `global` keyword in your code.
- **Do not have code outside of functions in `booking.py`:** The only code in `booking.py` should be function definitions. There should be no code outside of these functions in `booking.py`. Any code used to call or test these functions should be in another file such as `main.py`.
- **Follow good programming practices:** All included code must have a clear purpose and function. Including code that has no purpose, such as variables that are never used, may lead to marks being removed. You should understand each line of your program and each line should have a purpose. Marks may also be removed for other poor programming practices such as inappropriate recursion, redundant code, not making use of the specified functions when appropriate, or other code quality issues that demonstrate a lack of understanding.
- **Follow good style:** As much as possible, you should follow the PEP8 style guide. Both the autograder and your IDE should provide hints about style violations.
- **Python 3.12:** You code should work correctly in Python 3.12 and newer. These are the versions of Python the auotgrader will be compatible with.

- **No hardcoding:** You may not hardcode your returns, outputs, or writing to files. Your code should work correctly for any valid input. Attempts to fool or trick the auotgrader will result in the marker manually assigning a zero grade for that test.

▲ Back to Top

Assessment

The assignment will be marked as a combination of your auto-graded tests and manual grading of your code logic, comments, formatting, style, etc. Marks will be deducted for failing to follow any of the specifications in this document, not documenting your code with comments, using poor formatting or style, hardcoding, or naming your files incorrectly.

Marking Scheme:

- **Autograded Tests:** 80 points
- **Header comment including your name, student ID, username, creation date, and description of file:** 4 points
- **Descriptive in-line comments throughout code:** 4 points
- **Comment above each function describing what it does, its parameters, and its return value (if any):** 4 points
- **Meaningful variable names:** 2 points
- **Good programming practices *(see Non-Functional Specification)*:** 6 points

Total: 100 points

Note: *Marks from the autograded tests may also be reduced by the marker for violating any of the requirements given in the Non-Functional Specification, hard coding solutions, or failing to implement any features as specified.*

▲ Back to Top

Submission Instructions

You must upload **all files for all parts** (`booking.py` and `main.py`) to the Gradescope autograder. For a tutorial on how to use Gradescope to submit a code assignment, please review this link.

You may resubmit your assignment to Gradescope an unlimited number of times before the submission deadline. Gradescope will give you some feedback from the autograder each time you submit.

Any submissions after the due date will be considered late and subject to the late policy. Do not resubmit after the due date if you do not want to use late coupons.

Your final submission **must** include both the `booking.py` and `main.py` Python files, do not submit any `.csv` or `.tsv` files. Any missing files will result in a zero grade for that part of the assignment. Only your most recent submission will count for your assignment grade.

Submissions will not be accepted via e-mail or through any other means. The autograder will only accept files with the correct file extensions. Make sure all of your files have one file extension and that they are named correctly. Any incorrectly named files will result in a zero grade for that part.

Remember, **you are required to check that the files you submit are correct and all autograder tests are passing**. Teaching assistants will only review your submission for items relating to the Non-Functional Specification when marking, they will not give marks back for failed autograder tests.

▲ Back to Top

© 2026 Daniel Servos